

A Precision Audit of Robot Data-Quality Detectors

46 Static Checks Measured Against 20 Public LeRobot Datasets

Prabhu Gopal

26 August 2026 · bohrin 0.1.0 · commit cf39d78

Abstract

Static analyzers for robot demonstration data promise to catch defects that degrade a behavior-cloned policy before training begins. Their detectors are typically validated by fault injection: plant a known defect in a fixture, confirm the detector fires. That measures recall and cannot measure precision, because a detector that fires unconditionally passes every such test. We report the complementary measurement for *bohrin*, an open-source dataset linter: how often its 46 default detectors complain about data that other people collected, curated, published, and trained on. Scanning 20 public LeRobot datasets (4,342 episodes, 545,964 frames, 2–40 action dimensions, 5–50 Hz, simulated and real), all 20 parsed without error in 22.3s on a laptop CPU, yielding 189 findings (23 HIGH, 121 MEDIUM, 45 LOW). Twenty-seven detectors fired at least once, nineteen never fired, and four ever reached HIGH. One detector, `dynamics.inverse_residual`, reports HIGH on 50% of the corpus; we give three competing explanations, two of which are defects in the tool, and identify an ill-conditioned ridge solve as the first hypothesis to test. We also find that the default 300-episode triage cap changes no HIGH finding and two MEDIUM findings out of 189. We state plainly what this measurement does not establish: without ground-truth labels a fire rate is a rate of complaint, not a rate of correctness, and no policy was trained, so the premise that these defects degrade policies remains untested here. A secondary finding concerns the ecosystem rather than the tool: 18 of 20 datasets declare `robot_type: "unknown"`, undermining any embodiment-keyed calibration built on public metadata.

1 Motivation

A data-quality detector encodes a claim: that a specific pattern in the data predicts a specific training failure. Testing that claim by fault injection establishes recall and nothing else. A detector hard-coded to return HIGH passes every such test.

Precision determines whether such a tool is usable in practice. A HIGH severity that fires on most curated public datasets carries no information, and it breaks a `--fail-on HIGH` continuous-integration gate for every user. Measuring precision requires real data and a count of complaints. This report is that measurement.

2 Corpus and Method

Twenty LeRobot datasets were selected for breadth of embodiment and provenance rather than size. Fourteen are Open X-Embodiment datasets [1] that reached LeRobot through an RLDS conversion; six were published natively for LeRobot. Four are simulated. Action dimensionality spans 2 to 40 and control rates span 5 Hz to 50 Hz. Bohrin does not decode video, so the corpus occupies roughly 50 MB.

Embodiments were confirmed from published sources where possible: `asu_table_top` is a UR5 commanding joint velocities, `austin_buds_dataset` and `nyu_franka_play_dataset` are Franka arms [2], `cmu_stretch` is a Hello Robot Stretch [3], `d1r_edan_shared_control` is a wheelchair-mounted 8-DoF DLR arm [4], and `utokyo_saytap` is a Unitree Go1 quadruped [5], the one locomotion dataset in an otherwise manipulation-heavy corpus. Platforms not verified against a primary source are not claimed.

Each dataset was scanned twice. The **full** pass reads every episode and is the basis for every number reported here. The **default** pass is what a user gets from `bohrin scan`: a triage scan capped at 300 episodes, which subsamples 3 of the 20 datasets. Section 3.5 compares them. Both passes used no policy checkpoint, no target family, no calibration corpus, and no vision.

Table 1: Corpus summary. All twenty datasets resolved as `lerobot_v3`. OXE denotes an Open X-Embodiment dataset reaching LeRobot via RLDS conversion. H/M/L are counts of HIGH/MEDIUM/LOW findings from the full pass.

Dataset	Source	Hz	act	state	Episodes	Frames	Scan (s)	H	M/L
<code>ucsd_pick_and_place</code>	real, OXE	5	4	7	1,355	67,750	4.2	2	7/3
<code>xarm_lift_medium</code>	sim	15	4	4	800	20,000	1.6	1	6/2
<code>nyu_franka_play</code>	real, OXE	5	15	13	456	44,875	2.4	1	8/3
<code>utokyo_pr2_tabletop</code>	real, OXE	5	8	7	240	32,708	1.1	2	10/3
<code>pusht</code>	sim	10	2	2	206	25,650	0.7	0	3/4
<code>pusht_keypoints</code>	sim	10	2	2	206	25,650	0.9	0	3/4
<code>ucsd_kitchen</code>	real, OXE	5	8	21	150	3,970	0.6	1	7/2
<code>cmu_stretch</code>	real, OXE	5	8	4	135	25,016	0.8	3	8/1
<code>asu_table_top</code>	real, OXE	5	7	7	110	26,113	0.8	1	6/1
<code>dlr_sara_grid_clamp</code>	real, OXE	5	7	12	107	7,622	0.6	1	4/4
<code>dlr_edan_shared_control</code>	real, OXE	5	7	7	104	8,928	0.6	1	9/2
<code>dlr_sara_pour</code>	real, OXE	5	7	6	100	12,971	0.7	1	8/4
<code>aloha_mobile_cabinet</code>	real	50	14	14	85	127,500	2.1	1	5/1
<code>utokyo_pr2_fridge</code>	real, OXE	5	8	7	80	11,522	0.6	1	7/2
<code>tokyo_u_lsmo</code>	real, OXE	5	7	13	50	11,925	0.7	1	4/2
<code>aloha_sim_insertion</code>	sim	50	14	14	50	25,000	0.8	1	3/1
<code>austin_buds</code>	real, OXE	5	7	24	50	34,112	1.1	2	6/2
<code>unitreeh1_warehouse</code>	real	50	40	19	24	11,275	0.6	0	6/2
<code>utokyo_saytap</code>	real, OXE	5	12	30	20	22,937	0.8	1	5/0
<code>nyu_rot</code>	real, OXE	5	7	7	14	440	0.4	2	6/2
Total					4,342	545,964	22.3	23	121/45

Table 2: The four detectors that ever reported HIGH.

Detector	Fires	HIGH	Reading
<code>dynamics.inverse_residual</code>	95%	50%	Suspect (§3.3)
<code>stats.dead_dimension</code>	35%	35%	Plausible
<code>label.trajecory_label_mismatch</code>	20%	20%	Plausible
<code>multimodality.contradictory_actions</code>	10%	10%	Plausible

3 Results

3.1 Coverage and cost

All 20 datasets parsed, profiled, and reported with no crash and no adapter failure. Total wall-clock time was 22.3s on a laptop CPU with no GPU, median 0.8s per dataset. Seventeen of twenty produced at least one HIGH; none came back clean.

Wall time correlates with episode count at $r = 0.87$ and with frame count at only $r = 0.68$ (Figure 1), so per-episode fixed work dominates rather than raw data volume: `aloha_mobile_cabinet` reads 127,500 frames in 2.1s while `ucsd_pick_and_place` reads about half that in 4.2s with $16\times$ more episodes. Growth is sub-linear, with $97\times$ more episodes costing $10.5\times$ more time.

3.2 Selectivity

Of the 46 detectors that run in a default scan (48 registered, 2 held back; Section 4), **27 fired at least once** and **19 never fired**. Four ever reached HIGH (Table 2, Figure 3).

3.3 `dynamics.inverse_residual` fails its own audit

The detector fires on 19 of 20 datasets and returns HIGH on 10 of 20. A HIGH on half of a curated, published, widely used corpus is more likely a threshold defect than a real epidemic. Three explanations are consistent with the number, and this run cannot fully separate them.

The ridge solve is ill-conditioned. These scans emitted `LinAlgWarning: Ill-conditioned matrix (rcond =  $7.05\times 10^{-17}$ )` from `sklearn.linear_model._ridge`, alongside divide-by-zero, overflow, and invalid-

Scan cost tracks episode count, sub-linearly

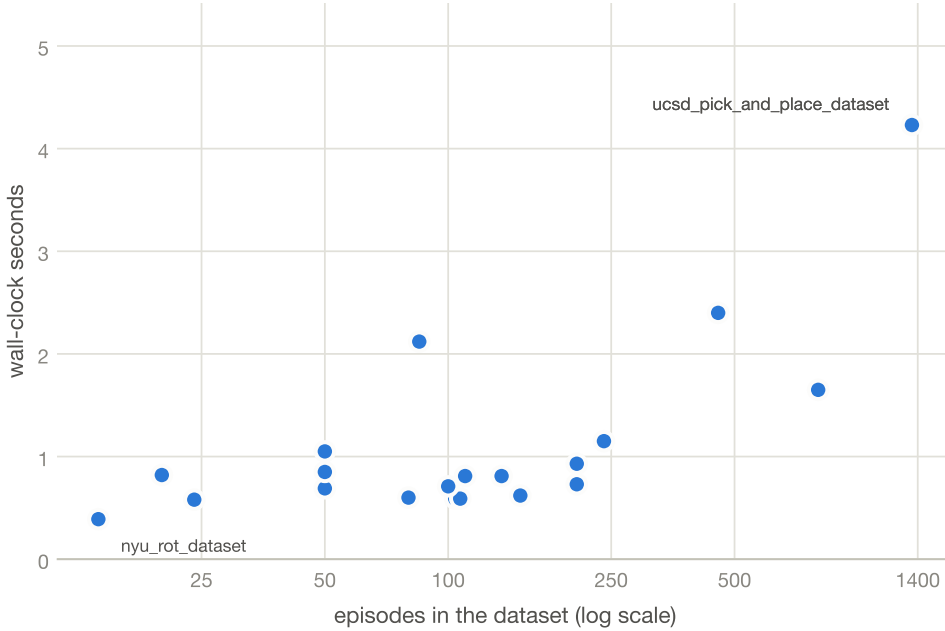


Figure 1: Scan wall-time against episode count ($\log x$). Cost tracks episodes ($r = 0.87$) more than frames ($r = 0.68$): bohrin reads Parquet columns and never decodes video, so per-episode fixed work dominates.

value warnings from `sklearn.utils.extmath`. The detector fits an inverse-dynamics model $g(o_t, o_{t+1}) \rightarrow \hat{a}_t$ and thresholds its residual; if the design matrix is near-singular the fit is unreliable and so is the residual. This is a concrete, checkable hypothesis and the first thing to test.

The threshold ignores control rate. Fourteen of twenty datasets run at 5 Hz. At 200 ms per step, consecutive states are only weakly related by the action between them, so an inverse-dynamics residual is legitimately large without any misalignment. A threshold calibrated on higher-rate data would over-fire here systematically.

The finding is real. Action/observation misalignment is a known hazard of format conversion, and 14 of the 20 datasets reached LeRobot through an RLDS conversion of an already-converted OXE dataset. Two conversion hops is a plausible place to lose synchronization.

The first two explanations are defects in the tool; the third is a defect in the data. Separating them requires the adjudication described in Section 5.

3.4 The nineteen silent detectors

Nineteen of the 46 default detectors produced no finding across 4,342 episodes. This is not evidence they are broken: several target defects this corpus should not have (`integrity.nan_inf`), the vision family had decoding disabled, and the entire `POLICY↔DATA` family stays silent without a checkpoint. It does mean those detectors carry no real-data evidence in this run, and their default severities rest on synthetic fixtures alone. It would be incorrect to describe this run as having validated 46 detectors.

3.5 Triage sampling does not change the conclusions

The default scan caps at 300 episodes, subsampling `ucsd_pick_and_place` (300 of 1,355), `xarm_lift_medium` (300 of 800), and `nyu_franka_play` (300 of 456). It reads 61% of episodes and 85% of frames in 21.5 s against 22.3 s.

Every HIGH finding is identical between the two passes, as are the counts of datasets scanned, datasets with at least one HIGH, detectors that fired, and detectors reaching HIGH. Two MEDIUM findings out of

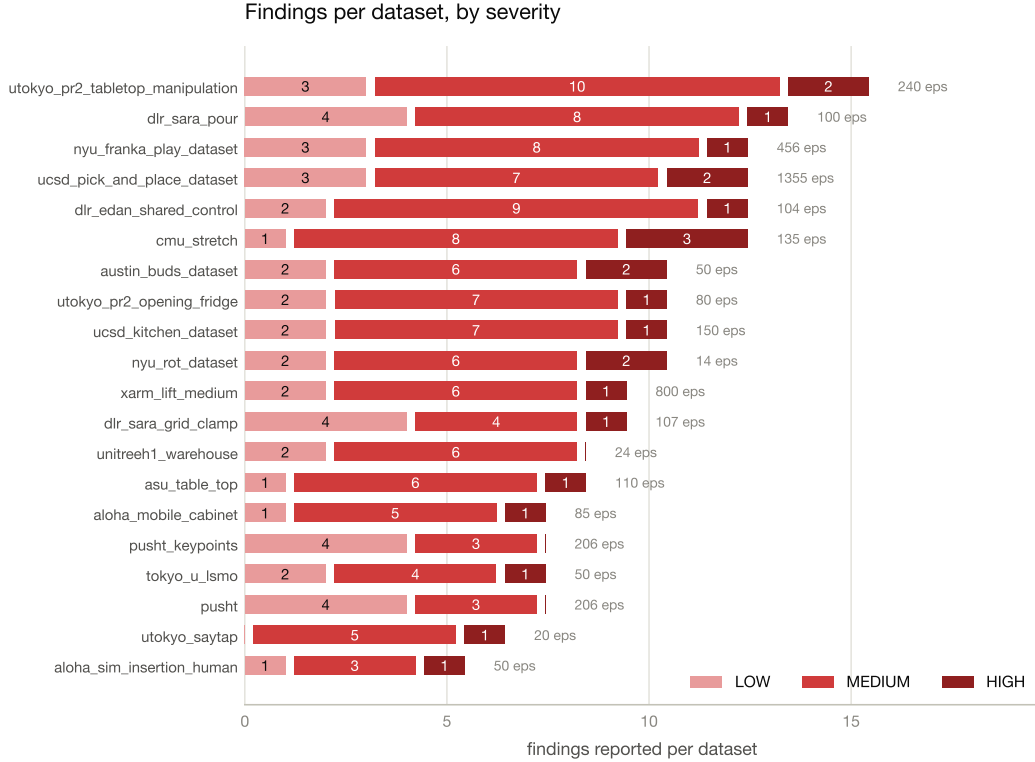


Figure 2: Findings per dataset, split by severity. Every dataset produced findings; 17 of 20 produced at least one HIGH.

189 differ (119 versus 121), and four detectors shift by one dataset each, all on the three subsampled datasets. Triage is a reasonable default on this corpus; that is a statement about these 20 datasets, not a general guarantee.

4 Two Detectors Were Correctly Held Back

Bohrin’s registry defines `DEFAULT_EXCLUDED`: detectors that are implemented and tested but withheld from a default scan pending recalibration, reachable with `--all`. It holds `smoothness.discontinuity_jump` and `integrity.declared_mismatch`, excluded after an earlier run of this harness measured them reporting HIGH on 70% and 60% of the corpus. Neither appears in any finding here, confirming the mechanism works. This is the project’s alternative to deleting a noisy detector: measure it, record the number, and gate it.

5 Limitations

These bound every number above.

Fire rate is not precision. Nothing here establishes that a single finding is correct. A 95% fire rate is equally consistent with “95% of public robot datasets have this defect” and with “the threshold is too tight.” No ground-truth labels exist for this corpus and none were constructed. Every rate reported here is a rate of complaint. The next step is to hand-adjudicate a stratified sample and publish a precision estimate with an explicit N .

No link to downstream policy performance. This is the most important gap. The premise behind every detector is that the defect it finds degrades a trained policy. This report does not test that premise: no policy was trained, no defect was ablated, no success rate was measured. The mechanisms are drawn from the literature, not from an experiment run here. A defensible causal claim requires training policies on data with and without a given defect and measuring the difference in rollout success. Until then,

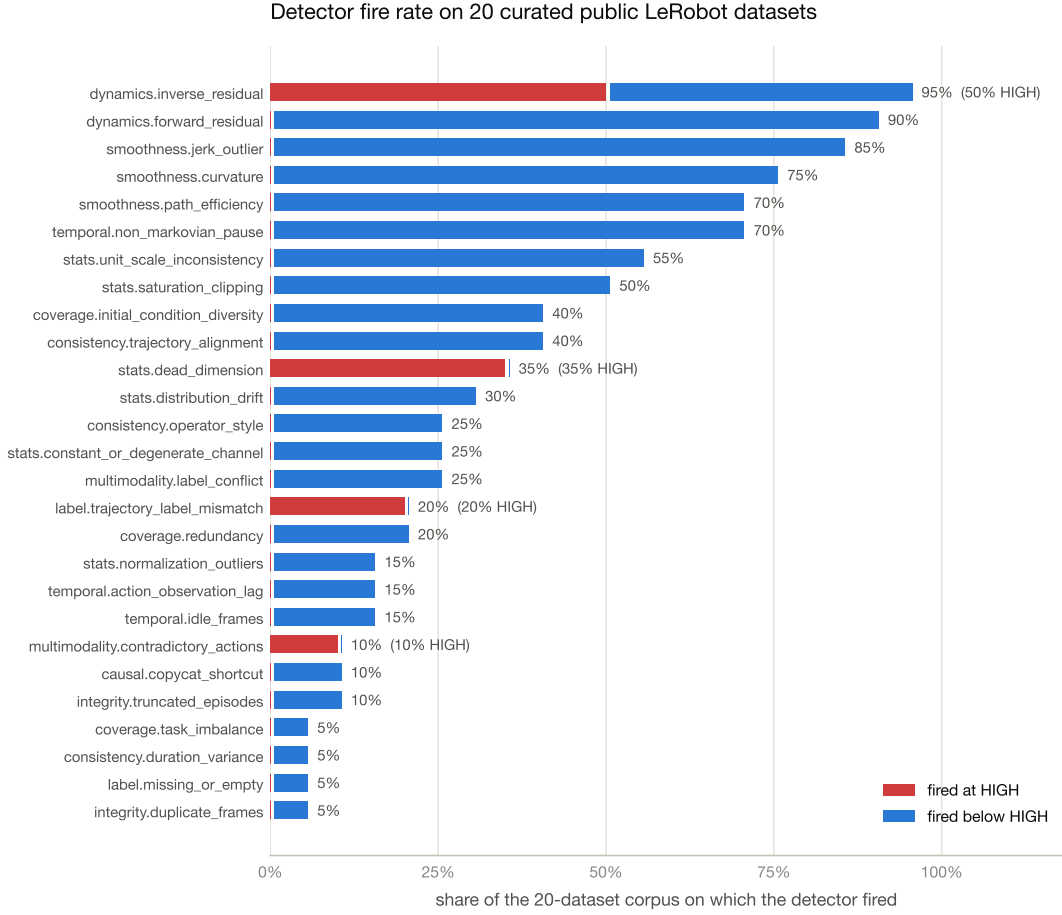


Figure 3: Per-detector fire rate across the corpus, with the HIGH share in red. Twenty-seven detectors appear; the other nineteen produced no finding on any dataset.

severity is a triage ordering rather than a prediction of harm.

Selection bias, in both directions. The datasets were chosen by the project’s own maintainer. They are curated and widely used, which biases toward clean data and makes a high fire rate more suspicious, the intended direction. They are also predominantly 5 Hz OXE conversions, so conversion artifacts are over-represented and native high-rate teleoperation is under-represented.

Single machine, single run. One laptop CPU with no GPU, one pass per configuration, warm cache. Timings are indicative rather than benchmarked: no repeats, no confidence intervals.

Default configuration only. No checkpoint, target, calibration corpus, or vision. The five POLICY↔DATA detectors were inert by construction.

The conformal gate never activated, and public metadata may prevent fixing that. Bohrin’s `--fpr` becomes a false-discovery bound only when a calibration corpus covers the dataset’s embodiment, using embodiment as a Mondrian category [6]. No corpus ships with the package, so every scan fell back to a robust- z heuristic. Filling that gap meets an obstacle this run surfaced incidentally: **18 of the 20 datasets declare `robot_type: "unknown"`**, with only the two ALOHA datasets naming an embodiment. A Mondrian taxonomy keyed on a field that 90% of this corpus leaves blank collapses to a single wildcard bucket and forfeits the group-conditional validity the design was chosen for.

Numerical warnings were observed and not resolved. An ill-conditioned ridge solve reaching the dynamics fit is a correctness concern, not cosmetic noise.

Dataset revisions are not pinned. A Hub-side update will change these numbers.

6 Next Steps

1. Test the ill-conditioning hypothesis: log the condition number of the design matrix per dataset and check whether it predicts the HIGH findings. If it does, the fix is regularization or abstention, not a new threshold.
2. Make the residual threshold control-rate aware, or verify that it already is.
3. Publish a precision estimate with an explicit N from hand-adjudicated findings.
4. Run a policy-outcome experiment on at least one detector, converting a mechanism argument into a measurement.
5. Resolve the embodiment-metadata gap before shipping a default calibration corpus.

Reproducibility

```
git clone https://github.com/prabhu-gopal/bohrin && cd bohrin
git checkout cf39d78
python -m venv .venv && .venv/bin/pip install -e '[dev]'
cd benchmarks/2026-08-26-lerobot-20-v0.1.0
../../.venv/bin/python scripts/run_sweep.py --out results/
```

Raw results, figure-generation code, and this document’s source are committed under `benchmarks/2026-08-26-lerobot-20-v0.1.0/`. Every figure derives from `results/sweep_full.json` alone; the committed JSON is authoritative if a figure and the text disagree.

References

- [1] Open X-Embodiment Collaboration. *Open X-Embodiment: Robotic Learning Datasets and RT-X Models*. arXiv:2310.08864, 2023.
- [2] TensorFlow Datasets catalog, Open X-Embodiment entries.
<https://www.tensorflow.org/datasets/catalog/overview>
- [3] S. Bahl et al. *Affordances from Human Videos as a Versatile Representation for Robotics*. CVPR, 2023.
- [4] J. Vogel et al. *EDAN: An EMG-controlled Daily Assistant to Help People with Physical Disabilities*. DLR Institute of Robotics and Mechatronics.
- [5] Y. Tang et al. *SayTap: Language to Quadrupedal Locomotion*. 2023.
- [6] V. Vovk, A. Gammerman, G. Shafer. *Algorithmic Learning in a Random World*. Springer, 2005.

Environment: Python 3.10.21, numpy 2.2.6, polars 1.44.0, scikit-learn 1.7.2, huggingface-hub 1.28.0, pydantic 2.13.4, rich 15.0.0. Figures: matplotlib 3.11.1, generated in a separate environment; matplotlib is a reporting tool and deliberately not a bohrin dependency.